센서 데이터 시뮬레이션

OOP · NumPy · Matplotlib

cchyun@gmail.com

Part 1. 전체 구조 설계

실습 목표 — 무엇을 만드는가?

- **OOP(객체 지향 프로그래밍)의 핵심 개념을 이해하고, NumPy와 Matplotlib으로 센서 데이터를 처리·시각화**

  - **추상화(Abstraction)**: 센서라는 개념을 클래스로 정의
  - **상속(Inheritance)**: 공통 설계도(부모 클래스)를 재사용하여 LiDAR · IMU 구현
  - **다형성(Polymorphism)**: 서로 다른 센서를 동일한 방법으로 처리

  - **실습 결과물**
    - LiDAR 2D 스캔 포인트 클라우드 시각화
    - IMU 3축 가속도 데이터 시각화
    - 두 센서를 하나의 루프에서 통합 처리

클래스 다이어그램 — 설계도 먼저 보기

- **로봇 시스템은 다양한 센서로 구성되며, 각 센서는 공통 동작(데이터 읽기)을 수행**

```
[ Abstract Class: Sensor ]
        ▲
        │ (Inheritance)
        ├──────────────────────────┐
[ Subclass: LiDAR ]        [ Subclass: IMU ]
- scan_range: float        - axis: int (3)
- num_samples: int         - frequency: int
+ get_data()               + get_data()
(Returns 2D points)        (Returns Accel/Gyro)
```

- **추상 클래스(Abstract Class)**: 직접 사용 불가, 자식 클래스가 반드시 `get_data()`를 구현해야 함
- **하드웨어 추상화 계층(HAL)**: 센서 종류가 바뀌어도 상위 코드를 수정할 필요 없음

단계별 구현 순서 — 5 Steps

- **Step 1**: `abc` 모듈을 사용하여 센서의 공통 인터페이스를 정의하는 **추상 클래스** 작성
- **Step 2**: LiDAR 센서 클래스를 상속받아 구현하고, **NumPy**로 거리 데이터 생성
- **Step 3**: IMU 센서 클래스를 상속받아 구현하고, **다차원 가속도 데이터** 생성
- **Step 4**: **다형성(Polymorphism)** 을 활용해 여러 센서 객체를 리스트로 관리하고 데이터 수집
- **Step 5**: **Matplotlib**을 사용하여 수집된 노이즈 데이터를 시각화

Part 2. 단계별 코드 구현

Step 1 — 추상 클래스 Sensor 정의

- `abc` 모듈의 `ABC`와 `abstractmethod`를 사용하여 모든 센서의 설계도(인터페이스) 정의

```
import numpy as np
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod

# 센서 추상 클래스: 모든 센서의 설계도 역할
class Sensor(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def get_data(self):
        """센서로부터 데이터를 읽어오는 추상 메서드"""
        pass
```

- `@abstractmethod` **데코레이터**: 자식 클래스에서 반드시 구현해야 하는 메서드 선언
- **직접 인스턴스 생성 불가**: `Sensor("test")` → `TypeError` 발생

Step 2 — LiDAR 센서 클래스 구현

- **LiDAR는 로봇 주변의 거리를 360도 방향으로 측정하며, NumPy 벡터화 연산으로 처리**

```python
# LiDAR 센서 클래스: 거리 데이터를 생성
class LidarSensor(Sensor):
    def __init__(self, name, num_samples=360):
        super().__init__(name)
        self.num_samples = num_samples

    def get_data(self):
        # 0~360도 각도 생성
        angles = np.linspace(0, 2 * np.pi, self.num_samples)
        # 기본 거리 5m에 가우시안 노이즈(평균 0, 표준편차 0.1) 추가
        distances = 5.0 + np.random.normal(0, 0.1, self.num_samples)
        return angles, distances
```

- `super().__init__(name)`: 부모 클래스의 생성자를 호출하여 `self.name` 초기화
- `np.linspace`: 0부터 2π까지 균등 간격으로 360개의 값을 생성
- `np.random.normal(0, 0.1, ...)`: 평균 0, 표준편차 0.1의 가우시안 노이즈

Step 3 — IMU 센서 클래스 구현

- **IMU는 가속도와 각속도를 측정하며, 고주파 데이터를 NumPy 배열로 처리**

```
# IMU 센서 클래스: 3축 가속도 데이터 생성
class ImuSensor(Sensor):
    def __init__(self, name, duration=100):
        super().__init__(name)
        self.duration = duration

    def get_data(self):
        # 시간축 생성
        t = np.arange(self.duration)
        # 3축(x, y, z)에 대한 가속도 데이터 생성 (랜덤 노이즈 포함)
        accel_data = np.random.randn(self.duration, 3) * 0.5
        return t, accel_data
```

- `np.arange(self.duration)` : 0부터 duration-1까지의 정수 배열 (시간 인덱스)
- `np.random.randn(duration, 3)` : shape이 (100, 3)인 2D 배열 — 행: 시간, 열: x/y/z 축
- `* 0.5` : 표준편차를 0.5로 스케일링

Step 4 — 다형성으로 센서 통합 관리

- **서로 다른 센서 객체를 하나의 리스트로 관리하고 동일한 방법으로 데이터를 수집**

```
def main():
    # 센서 객체 생성 (다형성 활용)
    lidar = LidarSensor("Hokuyo_LiDAR")
    imu = ImuSensor("InvenSense_IMU")
    sensors = [lidar, imu]

    # 시각화 설정
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

    for sensor in sensors:
        data = sensor.get_data()  # 타입에 관계없이 동일하게 호출
        ...
```

- **다형성 핵심**: `sensor.get_data()` 한 줄로 LiDAR와 IMU 모두 처리
- `plt.subplots(1, 2)`: 1행 2열의 서브플롯 생성 → `ax1`(LiDAR), `ax2`(IMU)
- `isinstance(sensor, LidarSensor)`: 객체 타입을 확인하여 시각화 분기 처리

Step 5 — Matplotlib으로 데이터 시각화

- 극좌표를 직교좌표로 변환하여 LiDAR 스캔을 2D로, IMU는 시계열로 표시

```
    if isinstance(sensor, LidarSensor):
        angles, dists = data
        # 극좌표 → 직교좌표(x, y) 변환
        x = dists * np.cos(angles)
        y = dists * np.sin(angles)
        ax1.scatter(x, y, s=5, c='blue', label=sensor.name)
        ax1.set_title("LiDAR Scan (2D)")
        ax1.set_aspect('equal')

    elif isinstance(sensor, ImuSensor):
        time, accel = data
        ax2.plot(time, accel[:, 0], label='X-axis')
        ax2.plot(time, accel[:, 1], label='Y-axis')
        ax2.plot(time, accel[:, 2], label='Z-axis')
        ax2.set_title("IMU Acceleration")
        ax2.legend()

plt.tight_layout()
plt.show()
```

전체 코드 — main() 완성

```
if __name__ == "__main__":
    main()
```

- `x = dists * np.cos(angles)`: 극좌표 `(r, θ)` → 직교좌표 `x = r·cos(θ)`
- `accel[:, 0]`: 2D 배열에서 0번째 열(X축) 전체를 슬라이싱
- `plt.tight_layout()`: 서브플롯 간 여백을 자동으로 조정
- `plt.show()`: 그래프 창 표시 — **반드시 마지막에 호출**

- **로봇 소프트웨어에서의 의미**
  - 이 `for sensor in sensors:` 루프가 바로 로봇의 **'Sense' 루틴**
  - 실제 로봇은 이 루틴을 수백 ms 단위로 반복 실행하며 환경을 인식

Part 3. 오류 해결과 심화 미션

자주 발생하는 오류와 해결법

단계	발생 가능 오류	원인 및 해결법
OOP	`TypeError: Can't instantiate abstract class...`	추상 메서드 `get_data`를 자식 클래스에서 구현하지 않은 경우. 메서드 이름을 확인하고 반드시 재정의해야 함
NumPy	`ValueError: operands could not be broadcast...`	배열 간 연산 시 크기가 맞지 않은 경우. `np.shape()`로 데이터 차원 확인 필요
Matplotlib	그래프 창이 뜨지 않거나 바로 닫힘	`plt.show()` 누락 또는 IDE 환경 설정 문제. 환경을 재점검해야 함
Common	`IndentationError`	파이썬은 들여쓰기가 문법의 핵심. 탭(Tab) 대신 4칸 공백(Space)을 일관되게 사용해야 함

심화 미션 1 — 이동 평균 필터 구현

- **NumPy를 활용하여 IMU 데이터의 노이즈를 줄이는 '이동 평균 필터' 작성**
  - **목표**: 필터링 전후 데이터를 비교 시각화

```python
def moving_average(data, window_size=5):
    # np.convolve를 활용한 이동 평균 필터
    kernel = np.ones(window_size) / window_size
    return np.convolve(data, kernel, mode='valid')

# IMU X축 데이터 필터링 및 비교
t, accel = imu.get_data()
filtered = moving_average(accel[:, 0])

plt.plot(t, accel[:, 0], label='Raw', alpha=0.5)
plt.plot(t[:len(filtered)], filtered, label='Filtered')
plt.legend()
plt.show()
```

심화 미션 2 — JSON으로 데이터 저장

- **수집된 센서 데이터를 `json` 모듈로 파일에 저장하고 다시 불러와 시각화**
  - **왜 JSON인가?**: 사람이 읽기 쉽고 파이썬 딕셔너리와 구조가 유사 → 로봇 설정값·로그 저장의 표준적인 방식

```python
import json

# 데이터 저장
t, accel = imu.get_data()
log = {
    "sensor": imu.name,
    "time": t.tolist(),
    "accel": accel.tolist()
}
with open("sensor_log.json", "w") as f:
    json.dump(log, f)

# 데이터 불러오기
with open("sensor_log.json", "r") as f:
    loaded = json.load(f)
print(loaded["sensor"])  # InvenSense_IMU
```

심화 미션 3 — 동적(실시간) 시각화

- `while` 루프와 `plt.pause()`를 사용하여 센서 데이터가 실시간으로 업데이트되는 시뮬레이션 구현
  - **의미**: 로봇의 **'Sense-Think-Act'** 루프를 코드로 모사

```
plt.ion()  # 인터랙티브 모드 활성화
fig, ax = plt.subplots()

while True:
    ax.clear()
    angles, dists = lidar.get_data()
    x = dists * np.cos(angles)
    y = dists * np.sin(angles)
    ax.scatter(x, y, s=5, c='blue')
    ax.set_title("LiDAR Real-time Scan")
    ax.set_aspect('equal')
    plt.pause(0.1)  # 100ms 대기 후 업데이트
```

- `plt.ion()`: Matplotlib 인터랙티브 모드 — `plt.show()` 없이도 즉시 화면 갱신
- `plt.pause(0.1)`: 0.1초 대기하는 동안 화면을 업데이트하는 핵심 함수

Part 4. Day3 연결 브리지

3일 전체 흐름 — 큰 그림

- **로봇 소프트웨어 개발의 단계: "기반 → 설계 → 확장"**
  - **Day 1 (기반 다지기)**
    - 가상환경(venv)과 Git으로 로봇 소프트웨어가 충돌 없이 동작하고 협업할 수 있는 **기반** 구축
  - **Day 2 (설계도 작성 — 오늘)**
    - OOP로 센서와 모터를 독립적인 부품(클래스)으로 모델링
    - **하드웨어 추상화 계층(HAL)**: 나중에 센서 종류가 바뀌어도 전체 코드를 수정할 필요 없는 설계
  - **Day 3 (실전 확장 — 내일)**
    - 데이터를 파일로 기록(JSON), 여러 작업을 동시에 처리(멀티스레딩)
    - **살아있는 시스템**처럼 동작하는 완성형 로봇 소프트웨어 구축

오늘 실습 요약

- **오늘 여러분은 로봇의 '감각 기관'이라는 부품을 튼튼하게 설계했습니다**
  - **OOP**: 추상 클래스 `Sensor` → `LidarSensor` · `ImuSensor` 상속 구현
  - **NumPy**: 벡터화 연산으로 360개 LiDAR 포인트, 3축 IMU 데이터를 한 번에 생성
  - **Matplotlib**: 극좌표 변환으로 2D 스캔 시각화, 시계열 가속도 그래프 표시
  - **다형성**: 서로 다른 센서를 하나의 루프에서 통합 처리
- **내일은 이 부품에 두 가지를 더합니다**
  - **기억력(저장)**: `sensor_log.json`으로 측정 데이터 영구 보존
  - **동시에 일하는 능력(멀티스레딩)**: 실제 자율주행 로봇과 유사한 완성형 시스템